



GATEWAY TECHNICAL

Supervised Learning: Classification



UNIVERSITY OF FLORIDA



1. An Introduction to Classification

Classification predicts discrete categories, not continuous values

Definition

Predict a categorical label (class) for each input

Regression vs. Classification

- **Regression:** Continuous output (e.g., house price = \$250,000)
- **Classification:** Categorical output (e.g., “spam” or “not spam”)

Goal

Learn a boundary that separates categories effectively

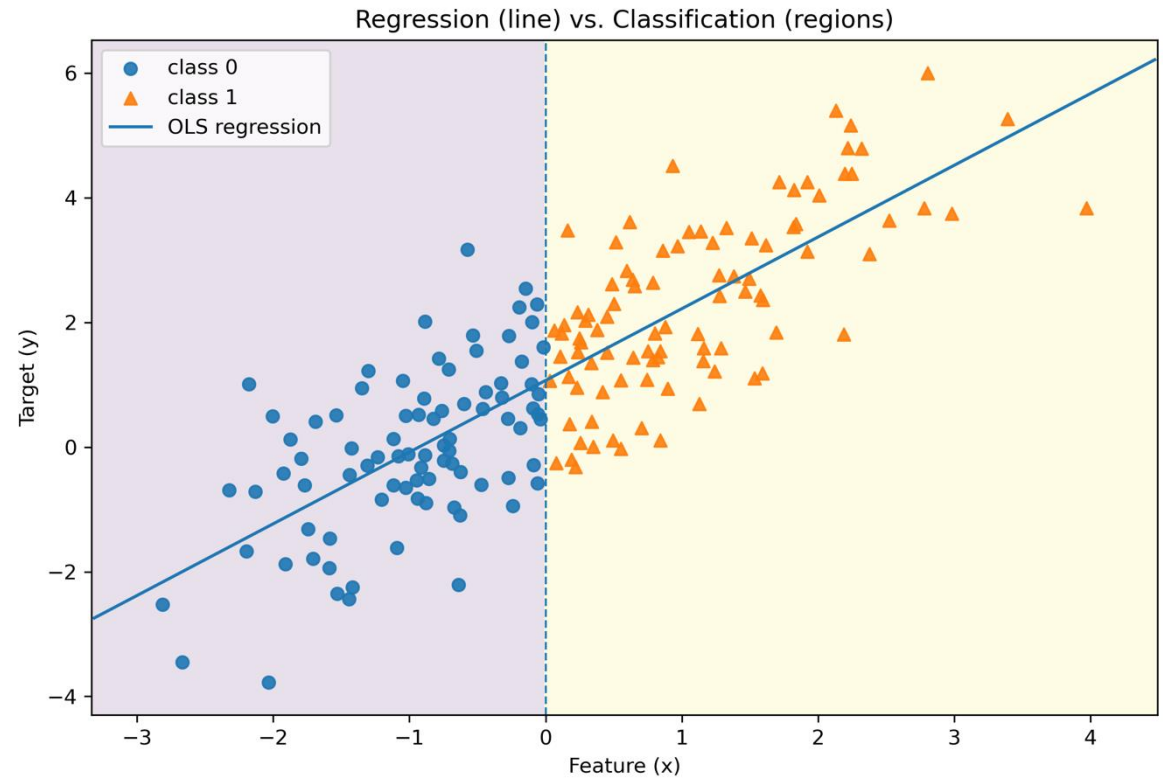


Figure: Simple 2D scatterplot showing regression as a line vs. classification as color-separated regions

Many real-world problems are classification tasks

Binary classification examples

- Fraud detection (fraud vs. not fraud)
- Customer churn prediction (churn vs. stay)
- Disease diagnosis (positive vs. negative)

Multiclass classification examples

- Numerical digit recognition (0–9)
- Wine quality prediction (low, medium, high)
- Document topic labeling (politics, sports, tech)

Key terminology for classification

Classes

The categories being predicted

Decision boundary

A line or surface that separates classes in feature space

Probability threshold

Converts predicted probabilities to labels (default = 0.5)

Confusion matrix

Evaluates how often predictions match true labels

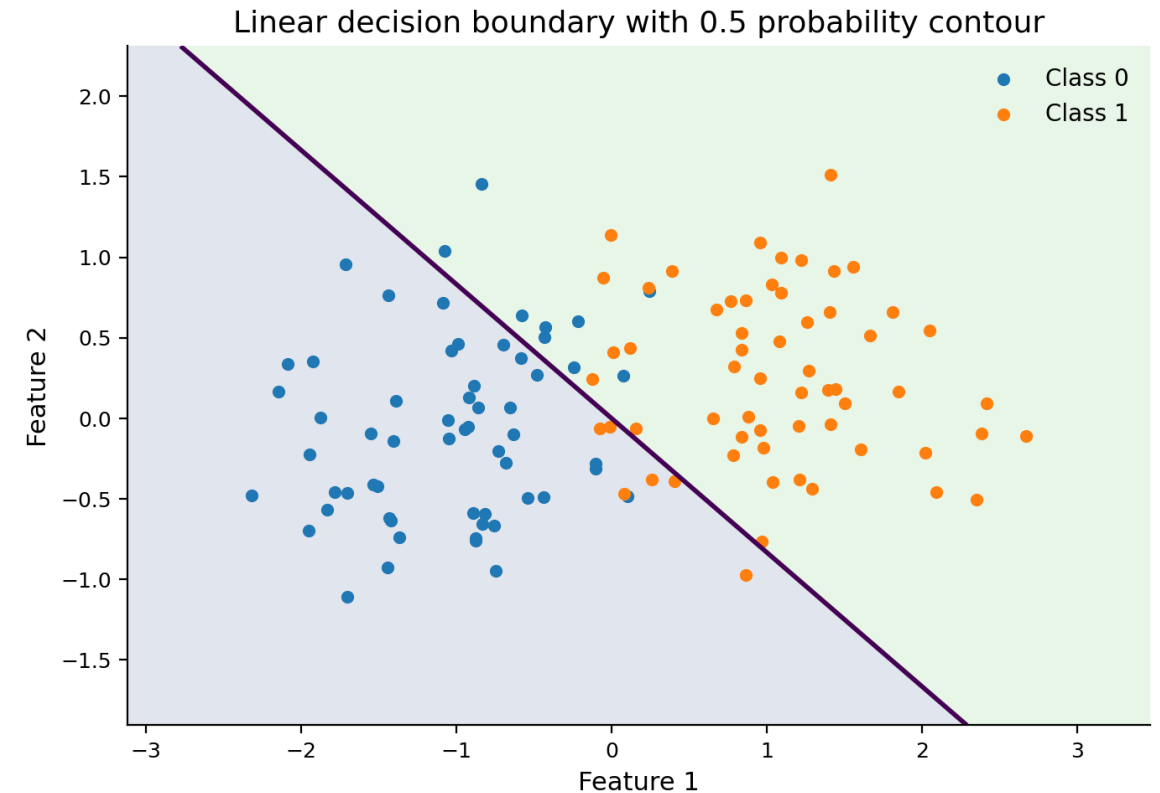
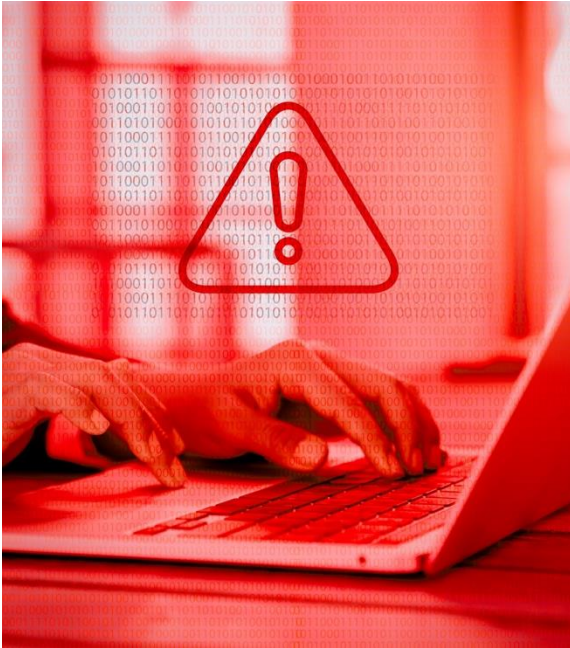


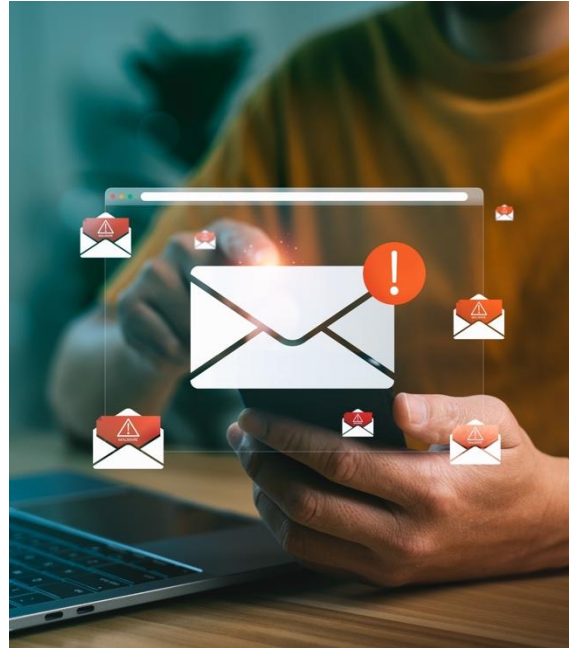
Figure: Simple 2D plot showing decision boundary separating two classes with a 0.5 probability line

Examples of classification problems



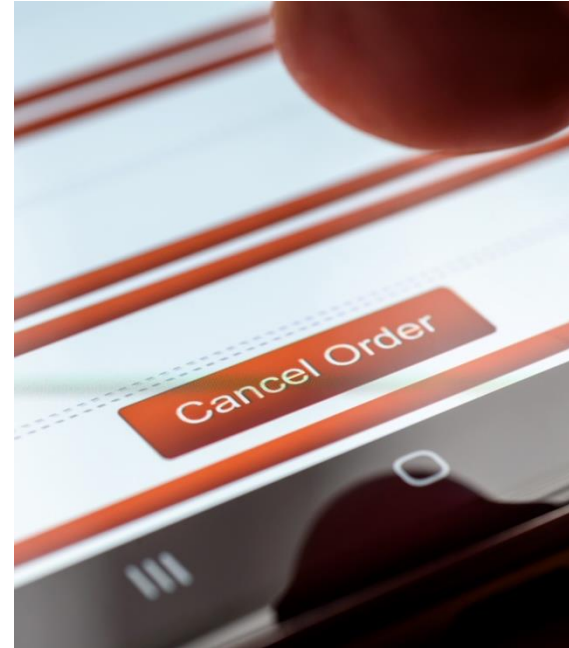
Fraud detection

Predict probability of a transaction being fraudulent



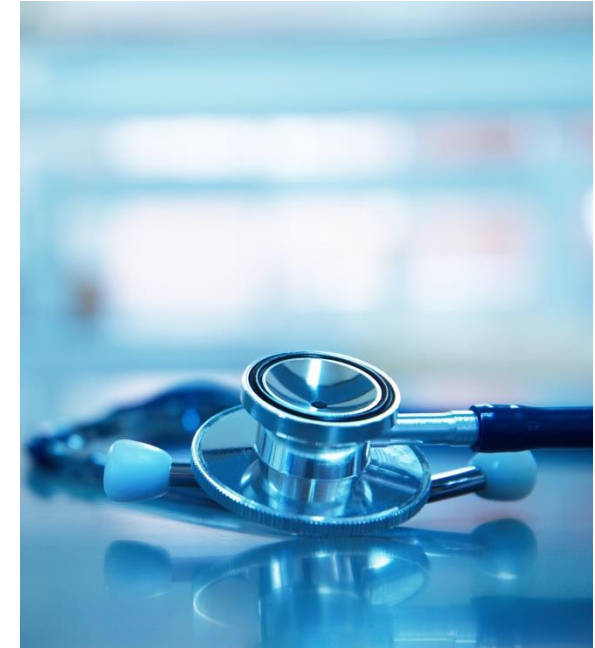
Message filtering

Classify messages as having a key characteristic or not



Group churn

Identify participants in a group most likely to leave (churn)



Medical diagnosis

Detect presence or absence of a condition

Why not just use linear regression?

Linear regression assumes **continuous, unbounded outputs**.

Categorical responses

Predictions can fall **below 0 or above 1**, making them invalid probabilities

Multiple classes

Numeric encoding imposes a **false ordering** (e.g., dog=1, cat=2)

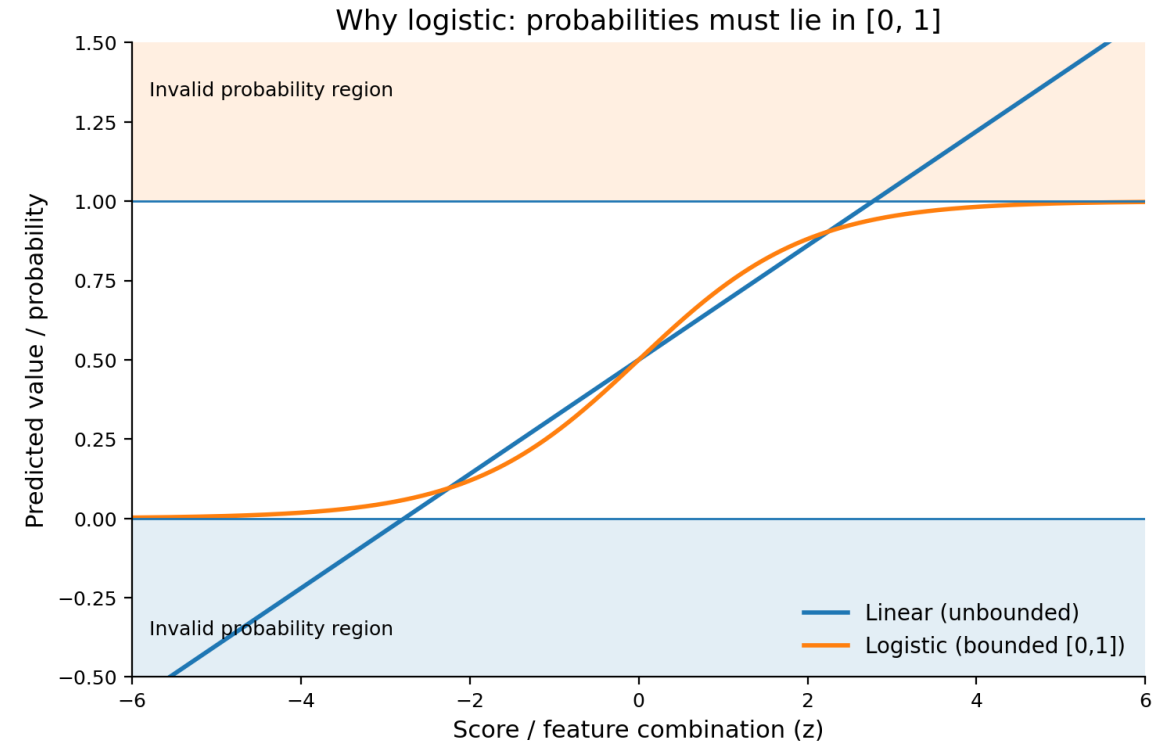


Figure: Comparison of regression line crossing probability limits vs. logistic S-curve constrained between 0 and 1



2. Logistic Regression

Logistic regression converts continuous scores into probabilities

- Regression predicts unbounded numeric values.
- Logistic regression uses the **sigmoid function** to squash predictions between 0 and 1:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

- Outputs interpretable probabilities for belonging to the “positive” class

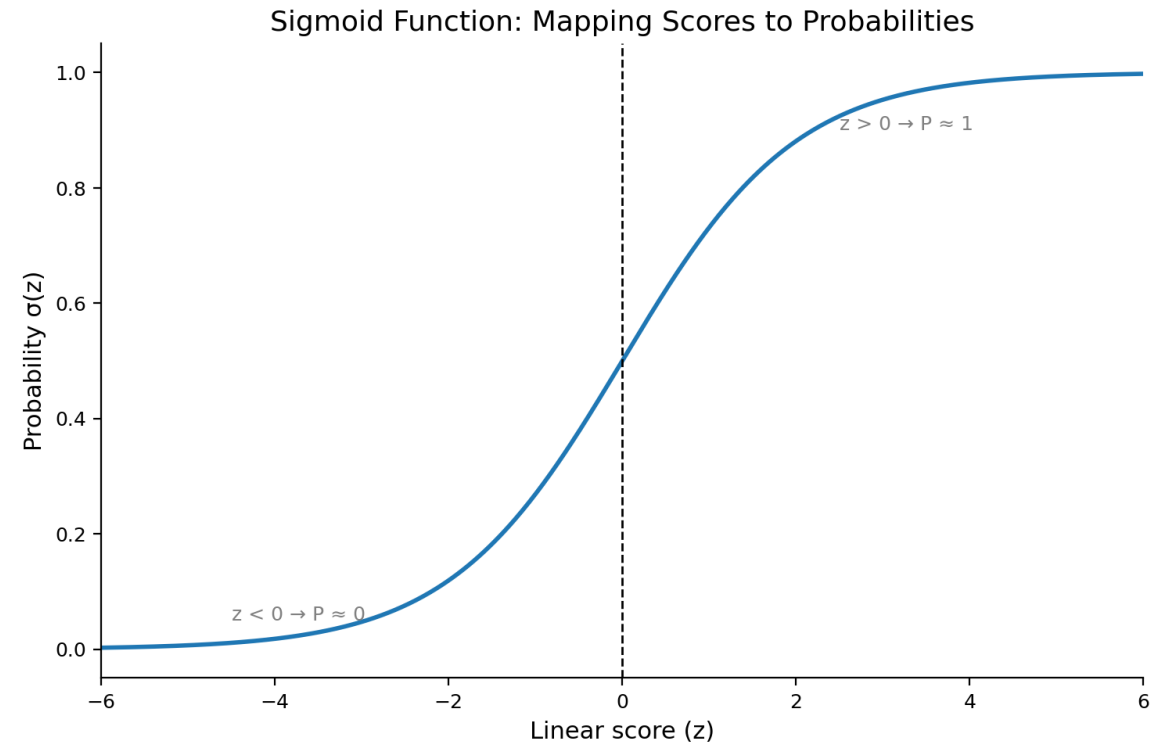


Figure: S-curve of the sigmoid function showing how $z < 0 \rightarrow \text{prob} \approx 0$ and $z > 0 \rightarrow \text{prob} \approx 1$

Logistic regression models the log-odds of the positive class

Core equation	$\log\left(\frac{p}{1-p}\right) = \beta_0 + \beta_1 x_1 + \dots + \beta_n x_n$
Odds	$p/(1-p)$ (e.g., 0.8 probability $\rightarrow$ odds = 4 : 1)
Log-odds	Each coefficient β_j represents change in log-odds per unit of x_j .
Exponentiating	e^{β_j} = multiplicative change in odds
Example	If $e^{\beta_1} = 1.5$ one-unit increase in x_1 multiplies odds by 1.5

Figure: Illustration of probability $\rightarrow$ odds $\rightarrow$ log-odds transformations

Logistic regression chooses parameters by maximizing likelihood

- Coefficients are chosen to make the observed outcomes most probable.
- Instead of minimizing squared error, logistic regression **maximizes likelihood** of correct labels.
- Guarantees valid probabilities within $[0, 1]$
- Leads naturally to measures such as deviance and log-likelihood

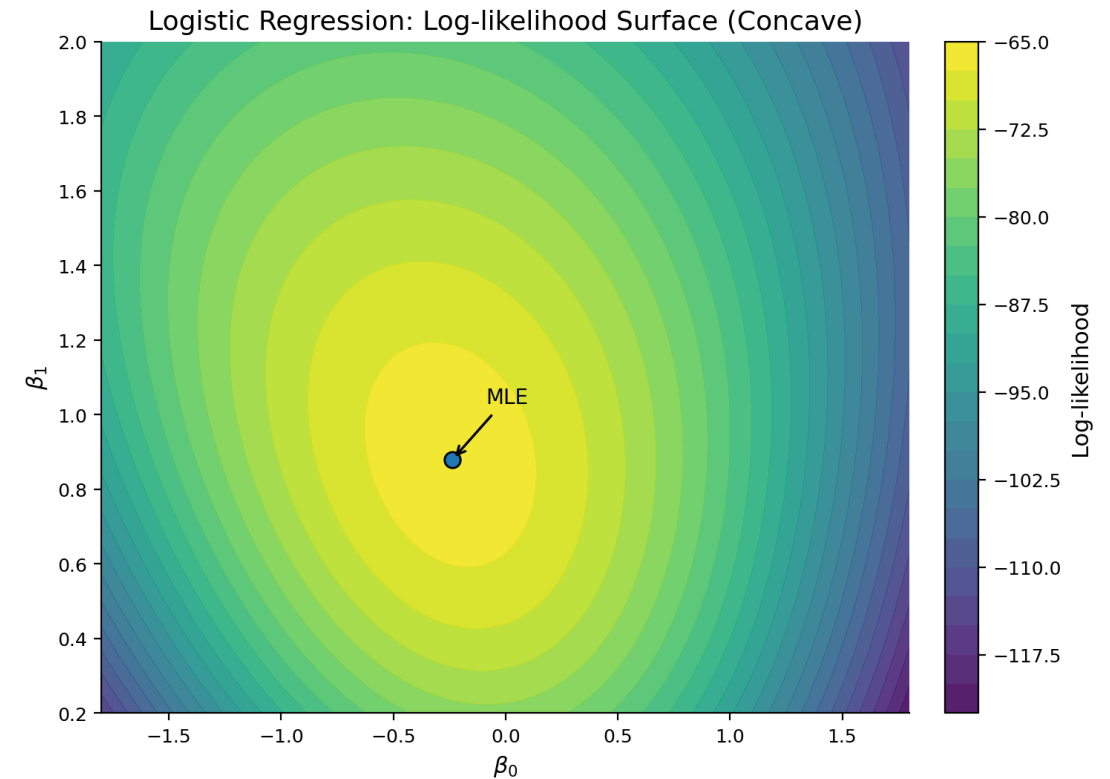


Figure: Conceptual diagram showing likelihood surface with single peak where parameters maximize probability of observed data

Probability thresholds turn model outputs into class predictions

Model produces probabilities p for each observation.

**Default
threshold**

0.5 $\rightarrow$ predict positive if $p > 0.5$

**Lower
thresholds**

More positives (higher recall)

**Higher
thresholds**

Fewer false alarms (higher precision)

Example

Fraud detection might use 0.2 to catch more fraud at cost of false positives.

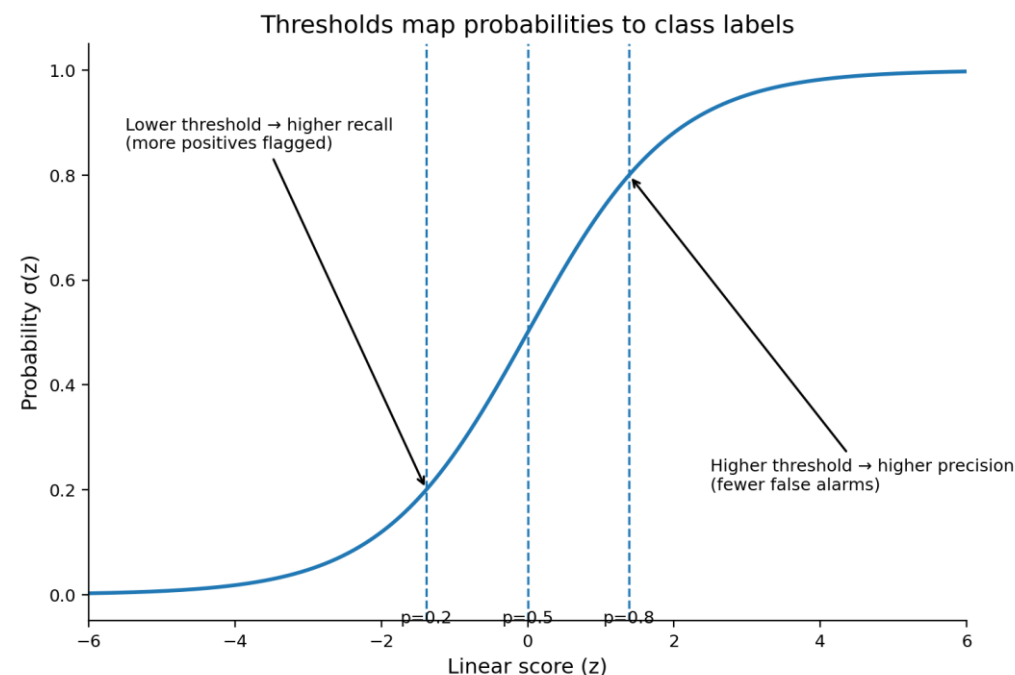


Figure: Sigmoid curve with varying cutoff thresholds showing how precision/recall trade-off shifts.

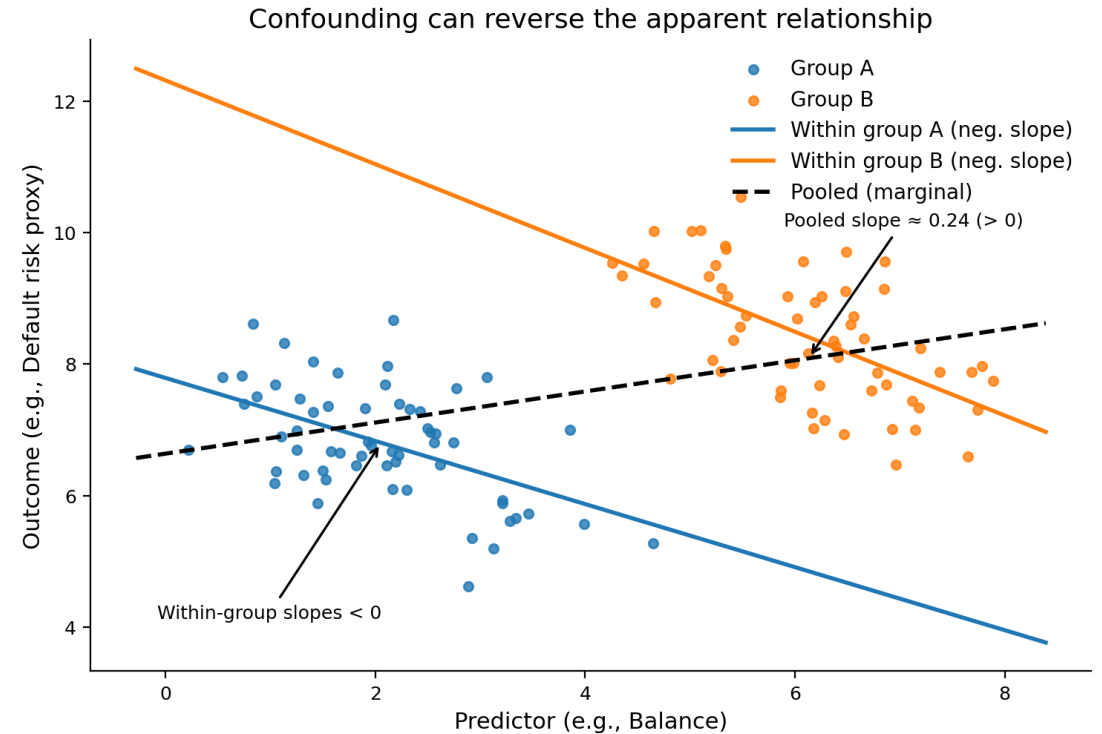
Confounding can flip or distort coefficient signs

- Adding correlated predictors changes coefficient size and even sign.

Key Idea

Coefficients reflect *conditional*, not marginal effects.

- Correlated predictors require careful interpretation.



Logistic regression assumes linear decision boundaries

- The model's decision boundary is **linear in the features**.
- In 2D → a straight line; in higher D → a hyperplane
- Performs well when separation is roughly linear
- Struggles with complex or curved class structures

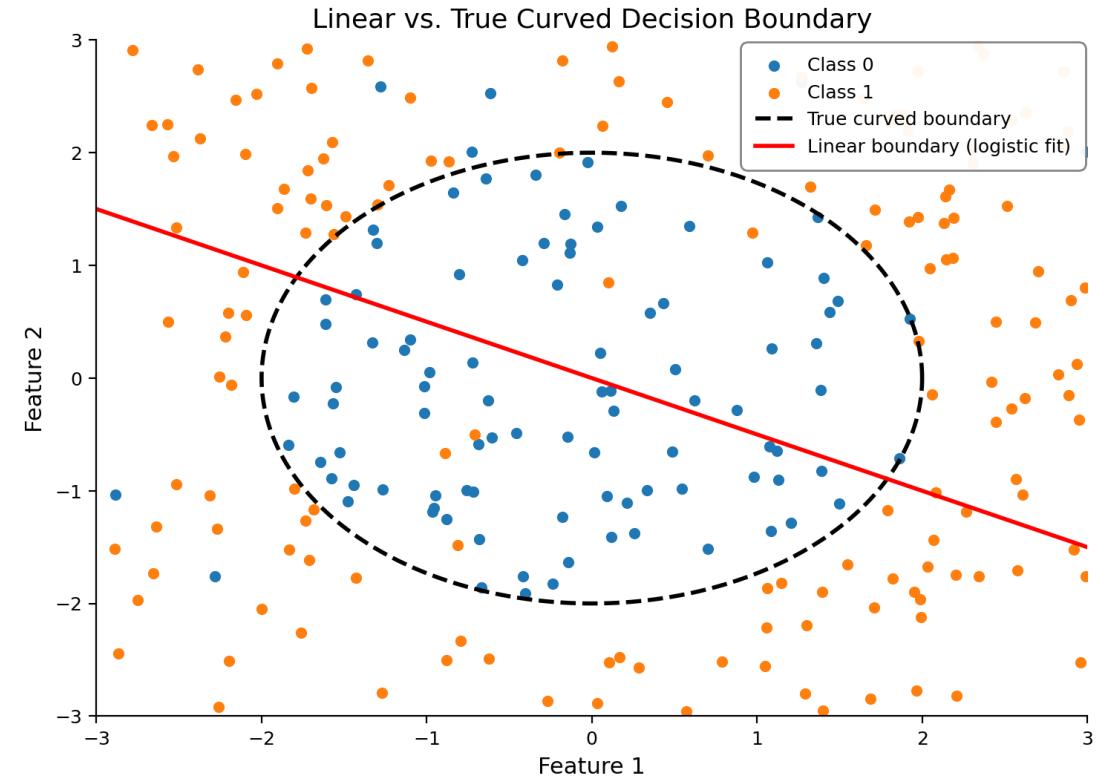


Figure: 2D plot comparing a linear decision boundary vs. a curved true separation

Logistic regression remains the “Hello World” entry way for classification



Advantages

Interpretable, fast, and robust

Provides a strong baseline before trying more complex models



3. Generative vs. Discriminative Approaches

Classification models can take two philosophical approaches

Discriminative models (e.g., Logistic Regression, Decision Trees)

- Model $P(Y|X)$ directly
- Focus on learning **boundaries between classes**

Generative models (e.g., LDA, QDA, Naive Bayes)

- Model $P(X|Y)$ and $P(Y)$, then apply Bayes' rule to compute
- $P(Y|X)$
- Focus on **how the data for each class is distributed.**

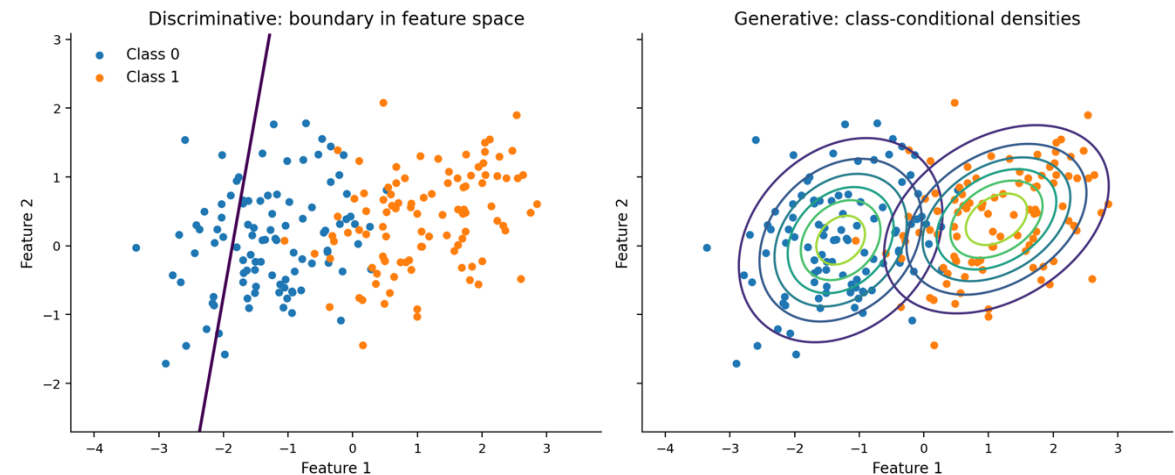


Figure: Illustration comparing discriminative decision boundaries (direct line between classes) vs. generative class distributions with overlapping densities

Why understanding the distinction matters

- **Discriminative** models often achieve **higher predictive accuracy**.
- **Generative** models capture **data structure** and handle **small-sample situations** better.
- When class covariances are equal, **logistic regression and LDA produce nearly identical boundaries**.
- **Choice depends on goal:** *prediction vs. understanding*.

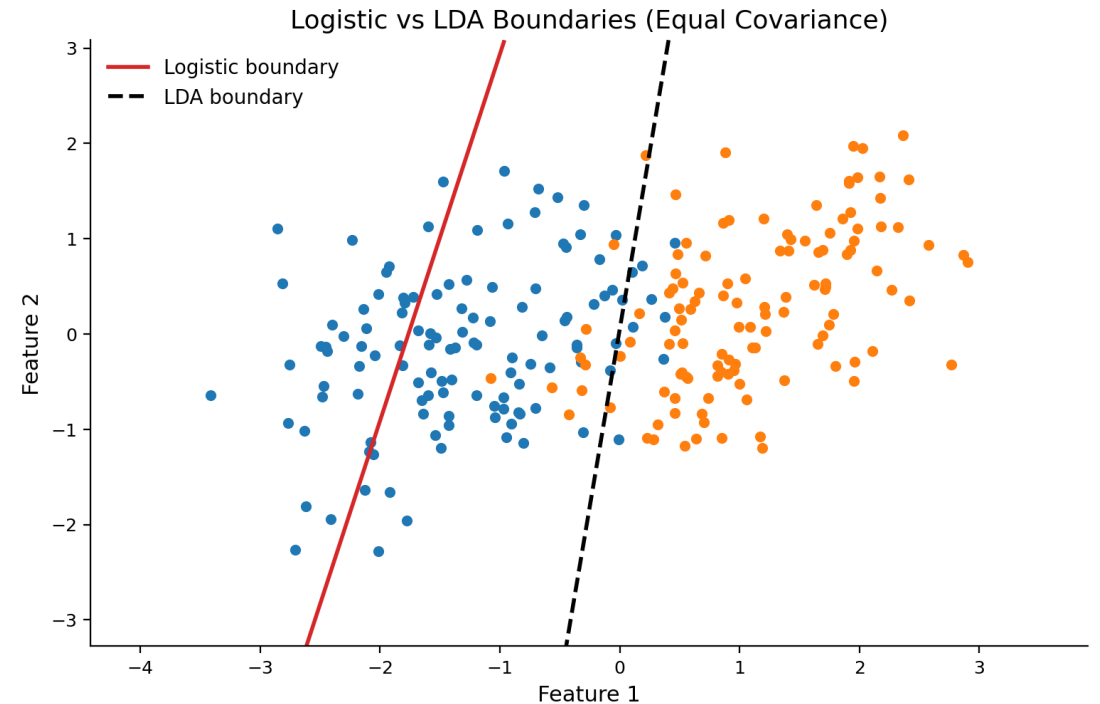


Figure: Side-by-side plot showing similar linear boundaries for logistic regression and LDA when covariances are equal



4. Decision Trees

Decision trees classify by asking a sequence of questions

Each path from root to leaf = **human-readable rule**

Root node

Starting point where data is first split

Decision nodes (branches)

Internal nodes that apply feature-based rules

Leaf nodes

Final outcomes or predictions

Example

“Age < 50?” → branch →
“Churn” or “Stay”

Figure: A Tree diagram showing nodes splitting by “Age < 50” and “Income < 40K.” Leaves (last boxes) are labeled with the predicted classes

Trees split data to increase node purity

Splitting metrics

Quantify how “mixed” classes are in a node

Gini impurity

$$1 - \sum p_i^2$$

Entropy

$$-\sum p_i \log(p_i)$$

Lower impurity → better separation

Check the default in your program if you are concerned. scikit-learn defaults to Gini, but both work similarly.

Class Proportions in Tree Nodes

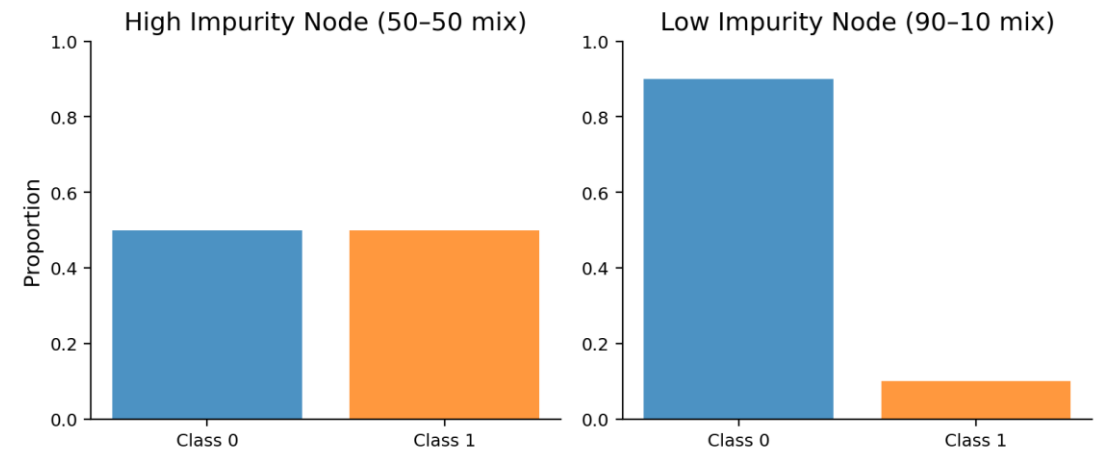
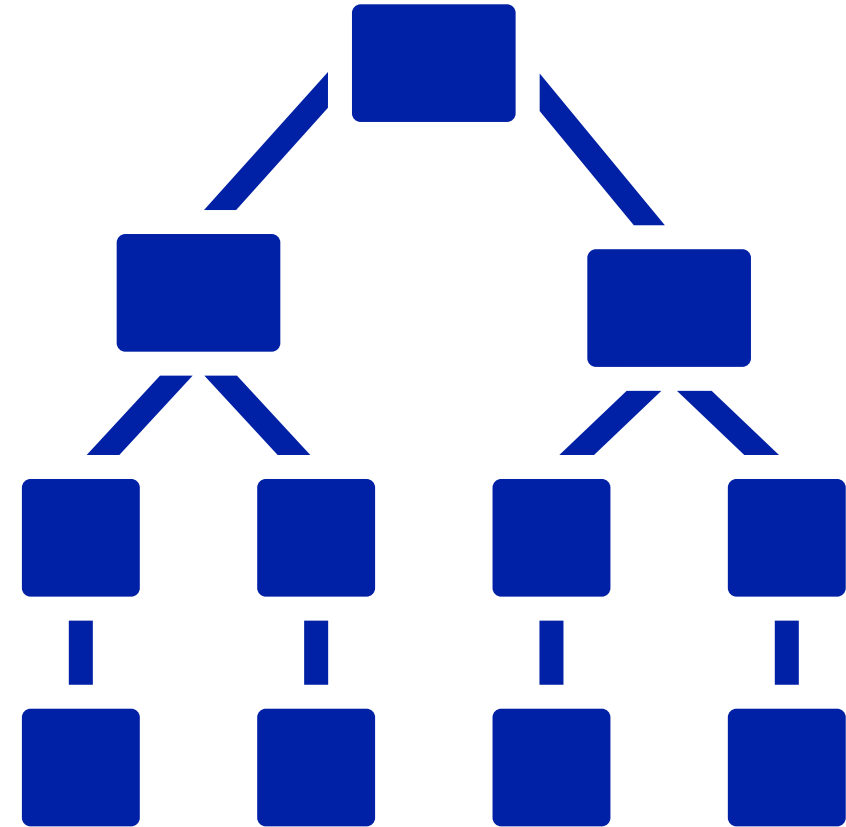


Figure: High-impurity vs. low-impurity node class proportions

Trees can overfit easily: Regularization controls complexity

- Deep trees can memorize training data → **high variance**.
- Shallow trees may underfit → **high bias**.
- **Key controls:**
 - `max_depth` — limit number of levels.
 - `min_samples_split`, `min_samples_leaf` — ensure each node has enough data.
- **Goal:** Find the sweet spot between underfitting and overfitting.



KNN offers a simple, nonparametric alternative for classification

- Predict a new point's class by **majority vote** among its K nearest neighbors.
- **Why it matters:** flexible, **no parametric assumptions**; adapts to local structure.
- It has limits, performance drops in **high dimensions** (curse of dimensionality); sensitive to feature scaling.

Key Idea

KNN bridges **parametric** (logistic) and **rule-based** (trees) models

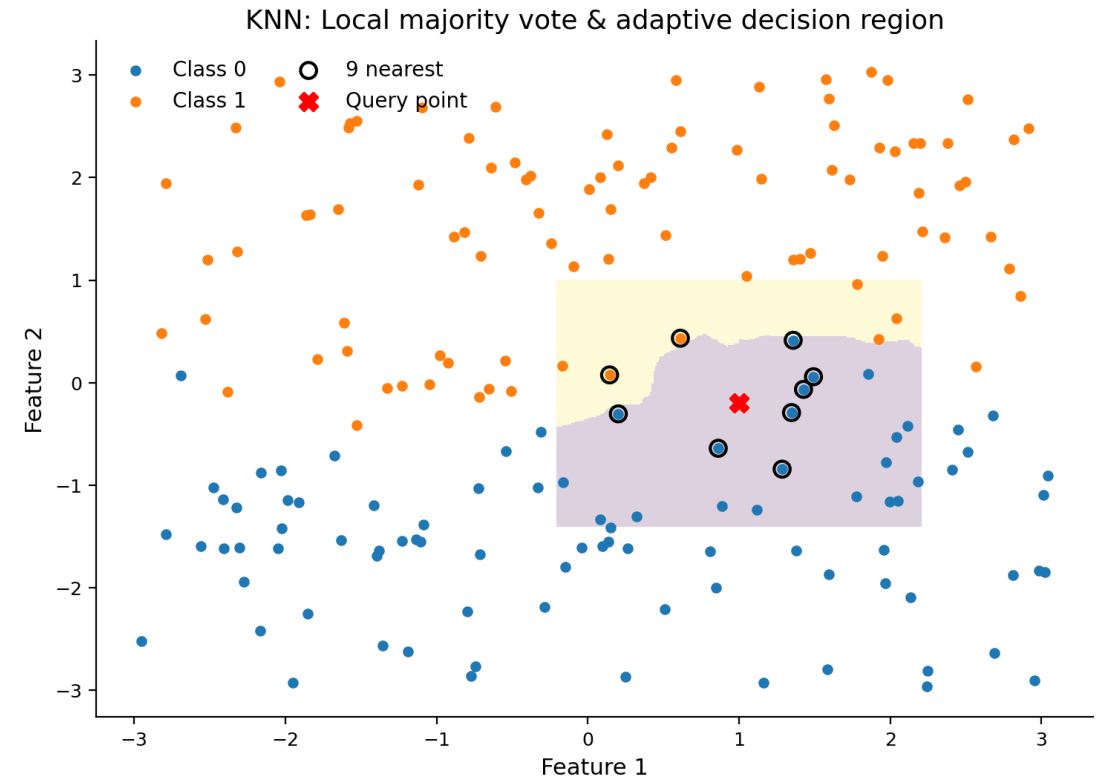


Figure: 2D scatterplot showing a query point classified by majority vote among its nearest neighbors; decision boundary adapts to local data

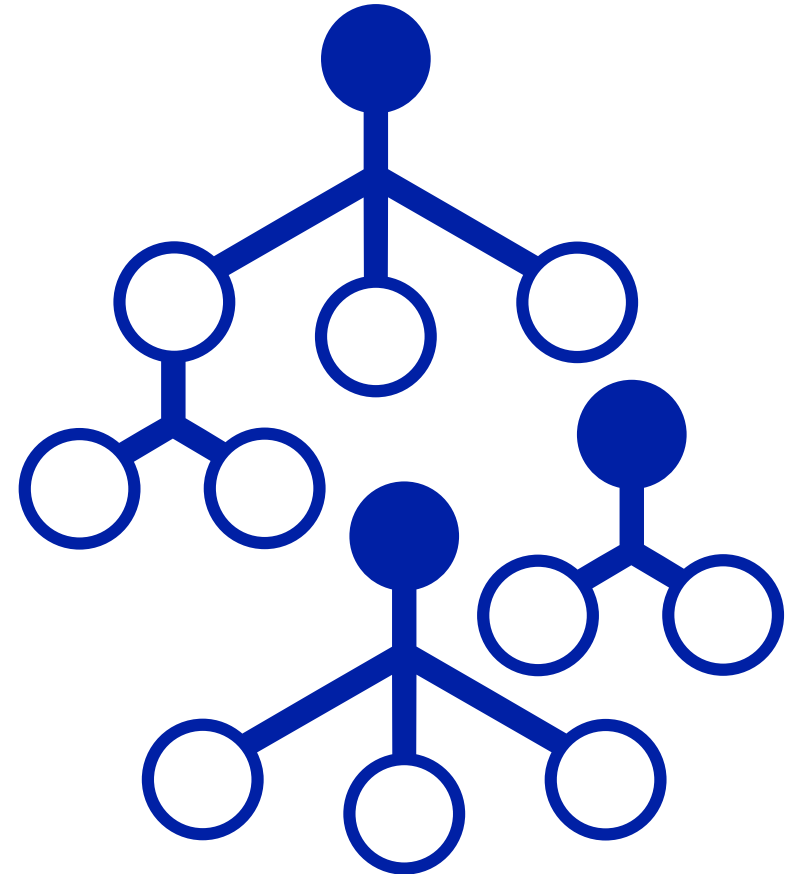


5. Ensemble Methods: Random Forest and XGBoost

Ensembles strengthen weak learners into robust predictors

- Single decision trees are **unstable** — small data changes can lead to big differences.
- Ensembles combine **many weak learners** to form a stronger, more stable model.

Benefits
• Reduce variance and overfitting • Improve accuracy and robustness • Generalize better to unseen data



Random Forest combines results from many decorrelated trees via majority vote (classification) or averaging (regression)

Adds feature subsampling

Each split uses only a random subset of features, reducing correlation among trees → more diverse models.

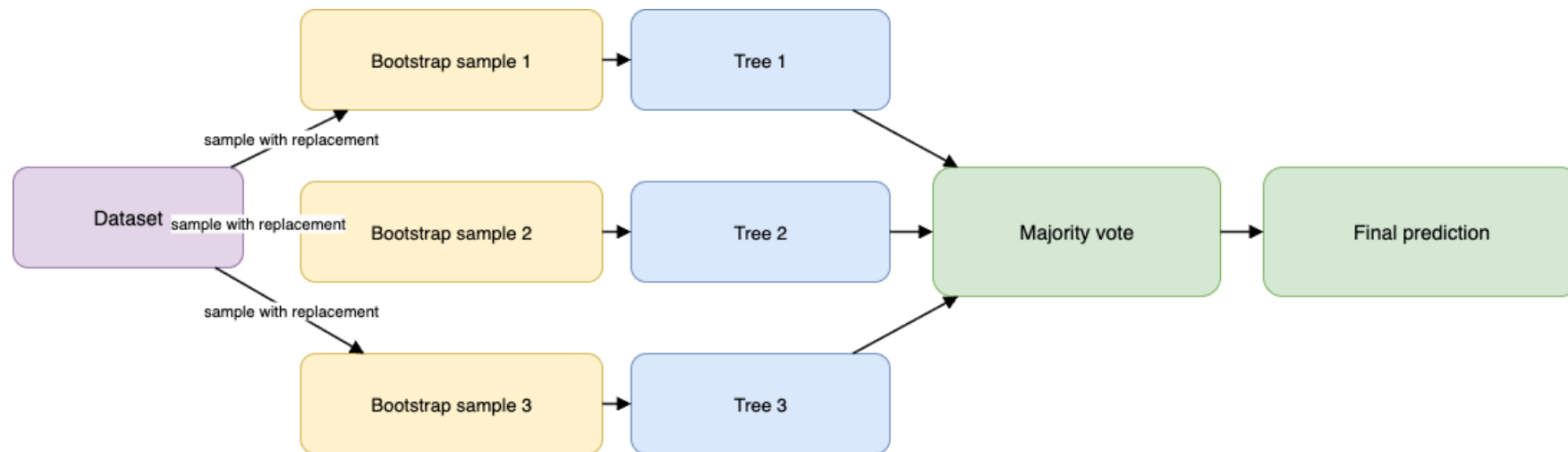


Figure: Multiple smaller trees trained on different bootstrap samples, voting on final class.
Effect: more stable, less prone to overfitting

Strengths and limitations of Random Forest

Strengths

- Reduces variance via bagging
- Handles nonlinearities and interactions
- Works well on tabular data
- Robust to noise and outliers

Limitations

- Less interpretable than a single tree
- Larger ensembles raise latency / memory
- Tuning still matters
(`n_estimators`, `max_features`)

XGBoost builds trees **sequentially**, each correcting errors of the previous



Figure: Illustration of sequential boosting — each tree focusing on the remaining errors of the previous one

Strengths and limitations of XGBoost

Strengths

- High accuracy on many datasets
- Regularization reduces overfitting
- Handles missing values
- Class weights for imbalance

Limitations

- Less interpretable (many small trees)
- Sensitive to hyperparameters
- Overfits without careful learning rate / depth

Random Forest vs. XGBoost: When to use each

Random Forest

- General-purpose, stable, strong out-of-box performance
- Low tuning effort, moderate interpretability

XGBoost

- Better for complex or imbalanced datasets
- Higher accuracy when tuned carefully
- Strong choice for performance-critical systems

Random Forest vs. XGBoost: Visual comparison



Figure: Comparison chart — RF: parallel averaging vs. XGB: sequential correction

Stability vs. Complexity: The ensemble trade-off

Random Forest

Increases **stability** through averaging

XGBoost

Increases **accuracy** through correction

Trade-off

- **Random Forest:** Simpler, interpretable, steady
- **XGBoost:** Powerful but complex and sensitive

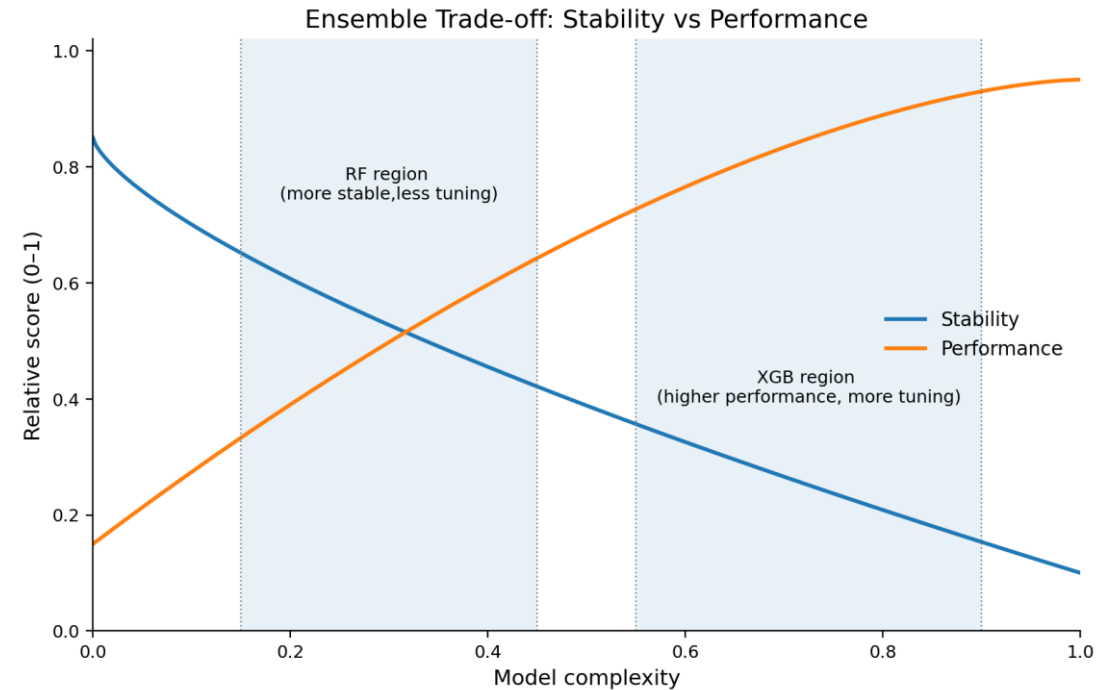


Figure: Two overlapping curves showing stability vs. performance trade-off



6. Evaluation Metrics for Classification

Accuracy alone rarely tells the full story

Accuracy

- % of correct predictions
- Works well only when classes are balanced

Example

95% accuracy sounds good, but in fraud detection (1% fraud), a model that always predicts “not fraud” achieves 99%.

Key idea

High accuracy $\neq$ good performance

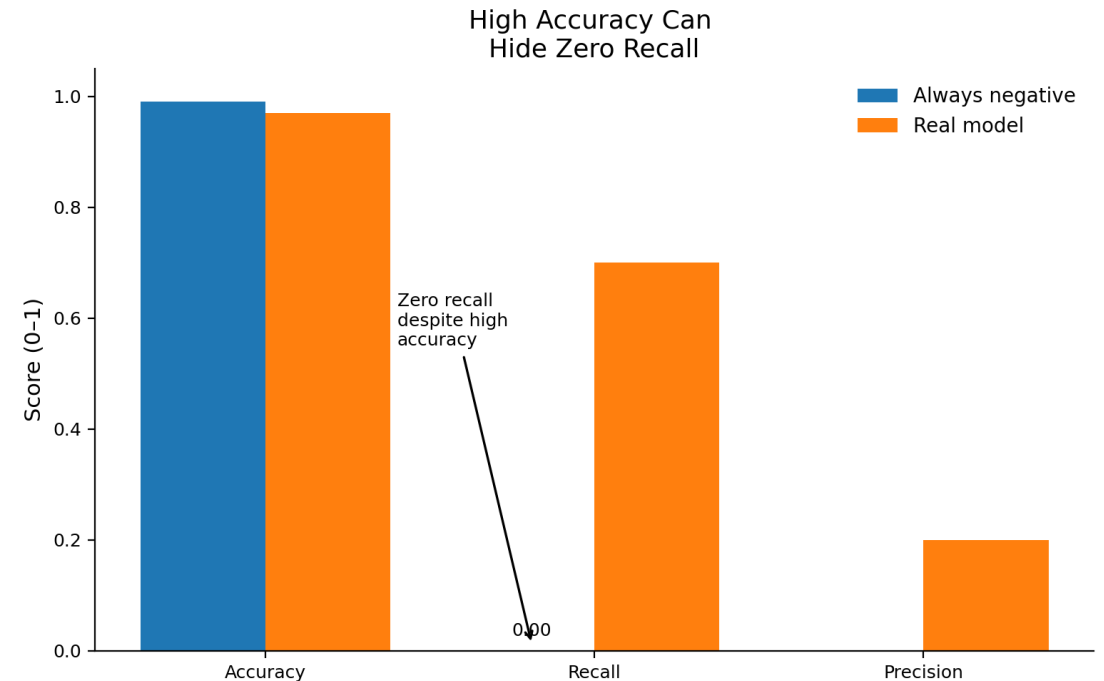


Figure: An “always negative” model achieving high accuracy but zero recall.

Precision, recall, and F1-score clarify performance trade-offs

Precision

- Of all predicted positives, how many are correct?
- **Formula:** "Precision" = $\frac{TP}{TP+FP}$
- Important when **false positives** are costly (e.g., flagging legit transactions)

Recall (Sensitivity)

- Of all actual positives, how many did we capture?
- **Formula:** "Recall" = $\frac{TP}{TP+FN}$
- Important when **false negatives** are costly (e.g., missing real fraud).

F1-score

- Harmonic mean of precision and recall.
- Balances the two when both matter.

The confusion matrix shows all four types of outcomes



ROC curves reveal model ranking quality across thresholds

ROC curve

Plots **True Positive Rate (Recall)** vs. **False Positive Rate (1 – Specificity)** across thresholds

AUC (Area Under Curve)

- Overall measure of ranking performance
- $AUC = 0.5 \rightarrow$ random guessing
- $AUC = 1.0 \rightarrow$ perfect classifier

Benefit

Threshold-independent evaluation

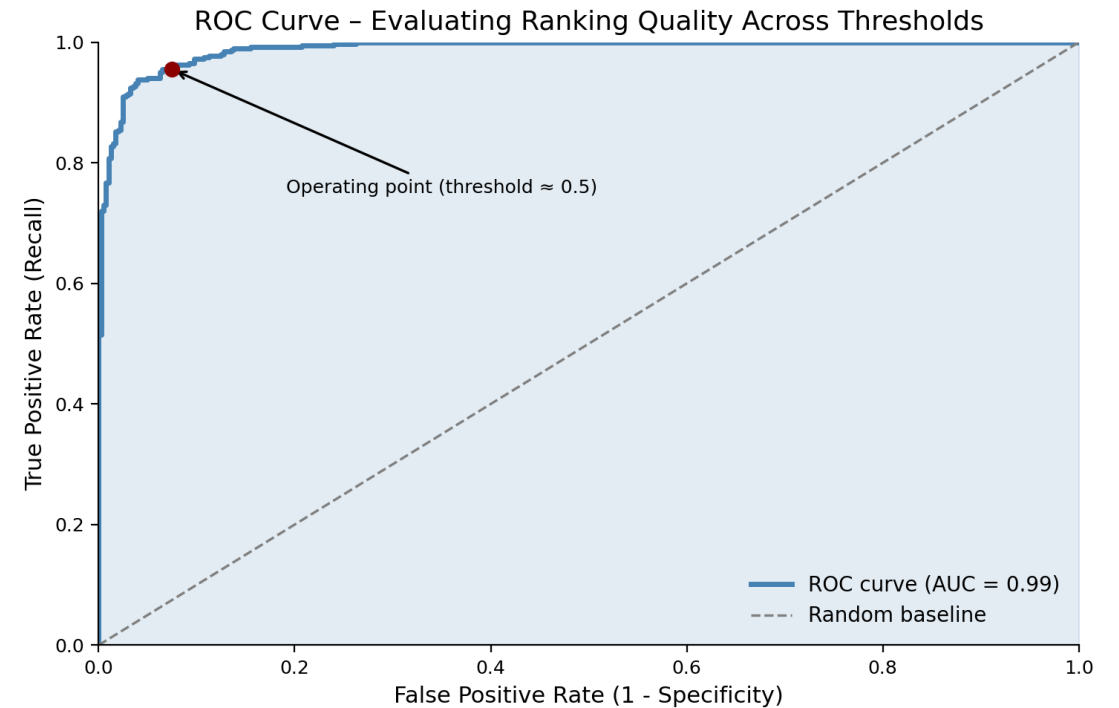


Figure: ROC curve with diagonal “random” line and high-performing curve above it

Precision–recall curve: Trade-off depends on chosen threshold

Lower threshold

Higher recall, lower precision

Higher threshold

Higher precision, lower recall

Example

Fraud detection → lower threshold to catch all suspicious cases

Visualization

Precision–recall curve

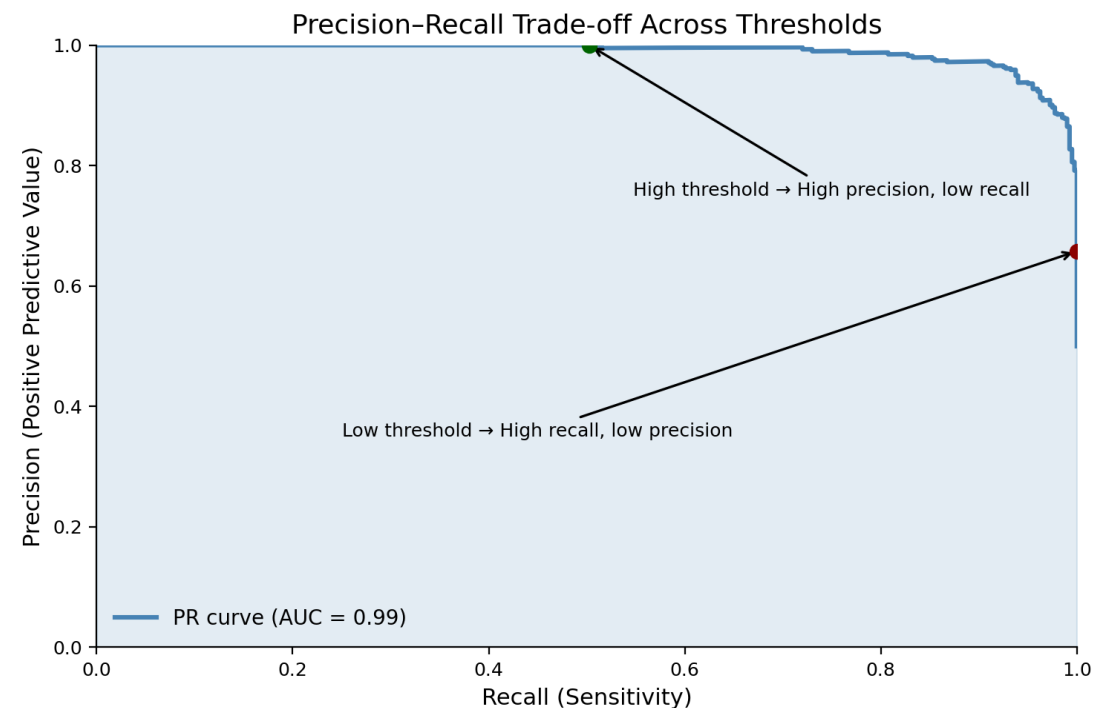


Figure: Precision–recall curves show how threshold movement changes trade-off

Matching metrics to real-world context



High precision preferred	Email spam filters (avoid blocking important mail)
High recall preferred	Medical screening (catch every possible case)
F1-score	Balanced need for both precision and recall
Key idea	Metric choice = context choice , not math choice

Cost-sensitive thresholding aligns models with goals

- Not all errors are equal — cost matters.

Example
• Missing fraud (false negative) costs more than false alarm (false positive). • Lowering threshold increases recall but increases false alarms.

- The decision rule is part of strategy, not just modeling.





7. Model Tuning and Validation

Cross-validation improves confidence in model performance



**Same principle
as regression**

Use multiple train/validation splits to assess generalization. Stratified k-fold preferred in classification (keeps class ratios consistent across folds.)

Figure: Diagram showing data split into folds, cycling through training and validation.

Classification models have key hyperparameters. Grid search systematically tunes these, testing for the best combination via cross- validation.

Decision Trees

`max_depth, min_samples_split`

Random Forest

`n_estimators, max_features`

XGBoost

`learning_rate, max_depth, subsample`

Trade-off

Computational cost vs. thoroughness

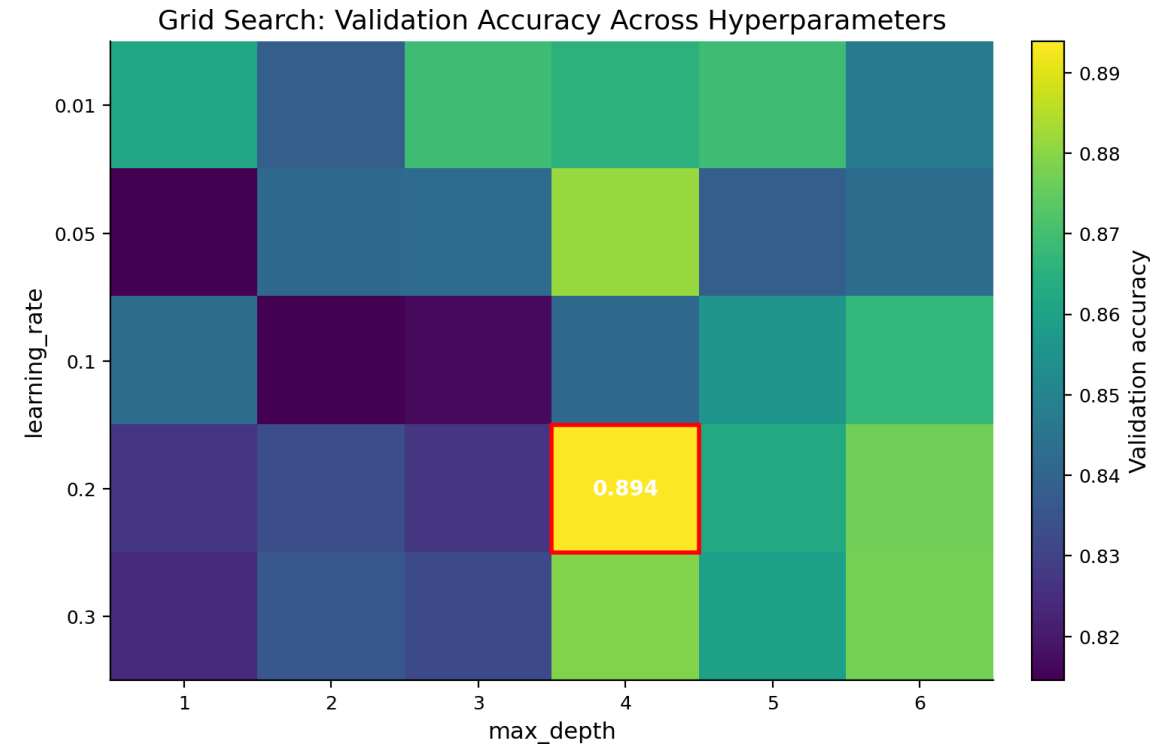


Figure: Grid search heatmap visual showing accuracy across combinations of hyperparameters.

Diagnosing overfitting, underfitting, and imbalance

Overfitting

- Training accuracy $\gg$ validation accuracy
- **Fix:** Reduce model complexity, use regularization, or gather more data.

Underfitting

- Both training and validation performance are poor
- **Fix:** Increase model complexity, add features, or try flexible models.

Class imbalance

- Model predicts majority class well but misses minority class.
- **Fix:** Use class weights, resampling, or metrics like F1 and AUC.

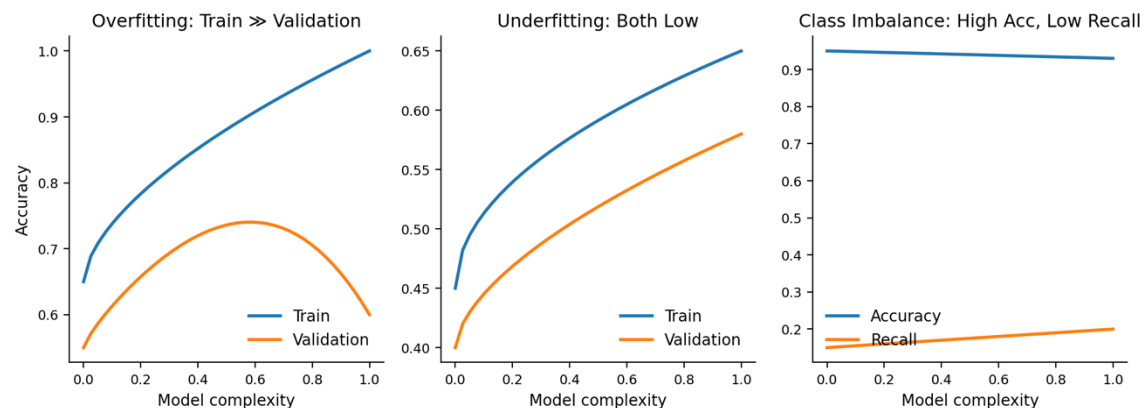


Figure: Three-panel chart showing:
(1) overfit: gap between train/test accuracy,
(2) underfit: both low,
(3) imbalance: skewed predictions.

Tuning is an iterative process, not a destination

- **Goal: Generalization**, not perfection
- Tuning is about **balancing**:
 - Complexity vs. interpretability.
 - Accuracy vs. robustness.
- No “best” model — only models that perform well **for a given context**





Hands-On Exercise:

Apply classification concepts end-to-end